OS ORANGE-2

Screenshots:

PHASE 1:

1A]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
saatg@SAATHVIK:~/os-u4-orange-problem$
```

1B]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
saatg@SAATHVIK:~/os-u4-orange-problem$
```

PHASE 2:

2A]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
saatg@SAATHVIK:~/os-u4-orange-problem$
```

2B]

```
saatg@SAATHVIK:~/PES1UG24AM238-pes-vcs$ [illegible]/89/5aaa3637a0865c5d8f77ee3f587dd19d3c043b989356dc6a528c1be2acd2b2 | head -20
00000000: 7472 2565 2039 3100 3130 3436 3434 3434 2064 tree 91.190644 f
[illegible]
00000020: [illegible] 7274 21fe 21fd ecb7 efb7 8183 8337 PES1UG24AM238
[illegible] 25de 576d b9eb b43b b430 3030 2070 2077 [illegible] 40000 [illegible]
[illegible] f142 8f32 ad6a 9970fe62 rc....Zo..[illegible]..b
00000060: 2883 71 (.q
```

PHASE 3:

3A&3B

```
saatg@SAATHVIK:~/os-u4-orange-problem$ ./pes init
echo "hello" > file1.txt
./pes add file1.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  commit.c
  untracked:  test_tree
  untracked:  test.txt
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  index.c
  untracked:  file2.txt
  untracked:  object.c
  untracked:  tree.h
  untracked:  commit.h
  untracked:  pes.c
```

```
saatg@SAATHVIK:~/os-u4-orange-problem$ ./pes add file2.txt
saatg@SAATHVIK:~/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  commit.c
  untracked:  test_tree
  untracked:  test.txt
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  index.c
  untracked:  object.c
  untracked:  tree.h
  untracked:  commit.h
  untracked:  pes.c

saatg@SAATHVIK:~/os-u4-orange-problem$
```

```
saatg@SAATHVIK:~/os-u4-orange-problem$ cat .pes/index
100644 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 1776759034 8 bye.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776758453 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776757868 6 file2.txt
100644 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776759034 12 hello.txt
saatg@SAATHVIK:~/os-u4-orange-problem$
```

PHASE 4:

4A]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ ./pes log
commit f6538d73877b8ca76c658e56f1eafcd9f97dd82f18349c8a1d4c62043af430de
Author: PES User <pes@localhost>
Date:   1776759317

    Add farewell

commit 6cf7ede2e5a11cb127b0bb62cd6e8f3925dc53913256b97df9b786ed11e0cf8e
Author: PES User <pes@localhost>
Date:   1776759317

    Add world

commit 12f912cdd23e7b4f7f919adadb031c4846c137e9e2fd2922e0a614c3049f1d28
Author: PES User <pes@localhost>
Date:   1776759317

    Initial commit

saatg@SAATHVIK:~/os-u4-orange-problem$
```

4B]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/12/f912cdd23e7b4f7f919adadb031c4846c137e9e2fd2922e0a614c3049f1d28
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6c/f7ede2e5a11cb127b0bb62cd6e8f3925dc53913256b97df9b786ed11e0cf8e
.pes/objects/94/4be1b9cbcf39b4de3ce97629261c20ae59d62c055c6b986a92acf9de63fdfa
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f4/119ab0e6733d85f46fab31c68077da71689d1c085e1a53211275ee09218096
.pes/objects/f6/538d73877b8ca76c658e56f1eafcd9f97dd82f18349c8a1d4c62043af430de
.pes/refs/heads/main
saatg@SAATHVIK:~/os-u4-orange-problem$
```

4C]

```
saatg@SAATHVIK:~/os-u4-orange-problem$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
f6538d73877b8ca76c658e56f1eafcd9f97dd82f18349c8a1d4c62043af430de
saatg@SAATHVIK:~/os-u4-orange-problem$
```

PHASE 5&6:

Q5.1 — Implementing `pes checkout <branch>`

Files that change in `.pes/`:

- `HEAD` is rewritten to `ref: refs/heads/<branch>`

- The index is replaced with the contents of the target branch's tree

Working directory changes:

- Read the target commit from `.pes/refs/heads/<branch>`, parse it to get the root tree, then recursively walk the tree and write each blob to its file path

- Files tracked in the current index but absent from the target tree must be deleted

What makes this complex: You need to reconcile three states — current tree, target tree, and the working directory — while handling directory creation/deletion. The operation also can't be fully atomic, so a crash midway can leave the repo in a broken state. Detecting dirty files (Q5.2) adds further complexity before the switch can even begin.

Q5.2 — Detecting a "Dirty Working Directory" Conflict

For each file in the current index, compute its blob hash from the working directory and compare it to the stored index hash. If they differ, the file has local modifications. Then check whether the target branch has a different hash for that same path. If both are true — the working directory diverges from the index, and the target branch also diverges from the index — checkout must abort, because proceeding would silently destroy the user's changes.

If the working directory is dirty but the target branch happens to agree with the current index, it's safe to proceed since the file won't be touched.

Q5.3 — Detached HEAD and Recovery

Detached HEAD means `HEAD` holds a raw commit hash instead of a branch reference like `ref: refs/heads/main`. This happens when you checkout a specific commit rather than a branch name.

Commits made in this state are written to the object store normally, but no branch pointer moves with them. Once you switch away, those commits have no named reference and become invisible to normal traversal. They won't be deleted immediately, but GC will eventually collect them.

To recover, if you still know the hash, create a branch pointing to it before GC runs. In PES-VCS, if the hash is lost and there's no reflog, the commits are effectively unrecoverable.

Q6.1 — Garbage Collection Algorithm

The approach is mark-and-sweep. In the mark phase, start from every file under `.pes/refs/heads/` and collect all branch tip hashes. For each commit, record its hash, follow its tree (recursively collecting all subtree and blob hashes), then follow its parent and repeat. Use a hash set to track visited objects so each is processed only once regardless of how many commits share it.

In the sweep phase, walk every file under `.pes/objects/`, reconstruct each object's hash from its path, and delete any that aren't in the reachable set.

For a repository with 100,000 commits and 50 branches, you visit all 100,000 commits. With heavy content sharing the unique object count might realistically be in the hundreds of thousands, all of which need to fit in the hash set during the mark phase.

Q6.2 — GC Race Condition with Concurrent Commits

A commit operation writes objects bottom-up: blobs first, then trees, then the commit object, and finally updates the branch reference. If GC runs its mark phase after the blob is written but before the commit object exists, the blob has no path connecting it to any branch. GC marks it as unreachable and deletes it. When the commit operation then tries to write a tree referencing that blob, the repository is corrupt.

Git sidesteps this with a time-based grace period: GC refuses to delete any object whose modification time is below a threshold (two weeks by default). A freshly written blob is always recent enough to survive, regardless of whether it's reachable yet. Git also enforces that objects are always written before the references that point to them, so any object a concurrent reader can see is either reachable or protected by the grace period.